Universität Bremen

Minimale Neuberechnung Abhängigkeits-Graph basierter Regeln zur prozeduralen Welten-Generation

Bachelorarbeit zur Erlangung des akademischen Grades
Bachelor of Science (B.Sc.) in Informatik

Eingereicht von

Maarten Behn

maarten.behn@uni-bremen.de

Prüfer

Prof. Dr. Gabriel Zachmann
Prof. Dr. Nico Hochgeschwender

FB3

Uni Bremen

SoSe 2026

11. March 2026

1 Einleitung

Prozedurale Welten werden mit Algorithmen generiert. Dabei werden werden erst grobe Eigenschaften generiert und diese dann mit immer feiner werdenden Details ausgeschmückt. Wenn der Generations Algorithmus in irgendeiner form angepasst wird kann es sein das eine vorher generierte Welt nicht mehr dem aktuellen Algorithmus entspricht. Die Rechenzeit eines prozeduralen Algorithmus steigt mit der Menge an Details die generiert wird. Bei komplexen Algorithmen die viele Details generieren, kann die lange Neugenerationszeit einer Welt beim iterativen entwickeln von generations Algorithmen stören.

1.1 Ziel der Arbeit

Ziel dieser Arbeit ist zu erforschen in wie weit die Neugenerationszeit einer prozeduralen Welt verschnellert werden kann, wenn nur die Aspekte der Welt neugeneriert werden die nicht mehr den neuen Algorithmus entspricht. Dazu habe ich ein System entwickelt in welchem der generations Algorithmus als Abhängigkeits-Graph dargestellt wird. Bei einer Neugeneration können zwischen Ergebnisse für Abschnitte des Abhängigkeits-Graph, welche sich nicht veränder haben wiederverwendet werden.

1.2 Aufbau der Arbeit

2 Stand der Technik

2.1 Prozedurale Generation

2.1.1 Noise & Zufälligkeit

- üblichster Weg
- Welt wird mit Zufälligkeit errechnet.
- Es gibt keine Regeln sonder die Regeln ergeben sich aus implizit aus dem Code
- Deterministisch nach Seeds

2.1.1.1 Nachteile

Es ist teils sehr schwierig sicherzustellen das komplexe Regeln eingehalten werden.

-> Fehlerhafte generation Möglich

ToDo:

L-Systems

2.1.2 Example based Model Synthesis

Eine spannende Arbeit im Bereich „constraint based generation“ ist Paul Merrels „Example-based Model Synthesis“. [1]

Die Idee des Algorithmus ist aus einem kleinen Input-Datensatz eine größere Struktur zu erzeugen die lokal den gleichen Regeln folgt.

Dabei besteht der Input-Datensatz aus einer Gitter Struktur, wo jede Celle im Gitter ein Wert zu geordnet wird. Nun wird eine Liste aller Nachbar Kombinationen erstellt die im Input Input-Datensatz vorkommen. Diese Liste beschreibt die Regeln nachdem ein neues Modell erzeugt wird.

Generations Algorithmus:

1. Dabei dem Gitter des neuen Modells in jeder Zelle alle Werte zugeordnet.
2. Wähle die Zelle mit den wenigsten Werten aus und entferne alle bis auf einen.
3. Entferne alle Werte aus den Nachbar Zellen für die eine Regel nicht mehr erfüllt ist.
4. Wiederhole 3. für jede Nachbar Zelle bei der Werte entfernt wurden.
5. Wiederhole 2. bis alle Zellen nur noch einen Wert enthalten.

2.1.3 Graph Grammars

2.1.4 Graph based Model Synthesis

ToDo:

Graph based Model Systemeis

2.1.5 KI basierte prozedurale Generation

Neuronale Netzwerke können genutzt werden um ähnliche Inhalte zu einem Trainings Datensatz zu erzeugen. In Arbeiten wie [2], [3] und [4] werden Neuronale Modelle genutzt um Levels aus bekannten 2D Spielen zu reproduzieren. Dabei wird in [2] wird ein Level

generations Model gegen ein Spieler Argent Modell trainiert, welches bewertet ob ein Level spielbar ist. Der Ansatz ist meistens fähig spielbare Levels zu generieren, wobei es vereinzelt zu generation Artefakten kommen kann. [2, S. 7]

Jedoch behandelt die Ansätze simple 2D Spiele, für die es viel vorhandene Level zu trainieren gibt. [2, S. 2]

ToDo:

Ist simple das richtige Wort?

In dem Ansatz wir außerdem gezeigt, dass Messbare Level Eigenschaften, wie die Menge der benötigten Sprünge genutzt werden können um „spannendere“ Level zu generieren. [2, S. 7]

[3]

- Analyse von der Möglichen validen Level eines 2D Mario spiels um unterschiedliche spannende Level zu erzeugen.

2.1.5.1 Terrain Diffusion

[5]

- Weiter Entwicklung von Noise Funktionen für Gelände generation.
- Es KI Modelle sind teilweise schneller als komplexe Noise Layer Systeme.
- Haben Geografische Erddaten als Input Datensatz verwendet.

2.1.5.2 Mit LLM Reden

[6]

- Aktuelle LLMs können sind fähig text basierte Darstellungen von Spiel Welten zu generieren und verstehen.
- Jedoch können sie nicht direkt mit Level Daten oder Tile Maps umgehen. Bzw dann generieren sie keine validen Ergebnisse

2.1.5.3 Umsetzung von generations Regeln in KI basierten Ansätzen

Bei der prozeduralen generation mit KI Systemen werden generations Regeln meist nicht explizit definiert. Sie ergeben sich aus dem Trainings-Datensatz und den Bewertungs Methoden der Modells in „Adversarialen“ Ansätzen.

ToDo:

Sollte ich KI sagen oder genauer sein?

Nachhaltig sicherzustellen, dass alle erwünschten Generationsregeln eingehalten werden, bleibt ein Bereich in dem Aktiv geforscht wird.

2.1.5.4 Bewertung

KI basierte Ansätze zur prozeduralen Generation sind ein sich aktiv entwickelnder Bereich indem

- Kann nur reproduzieren was schon vorhanden ist bzw benötigt größere Mengen an Input Daten.

-> Schränk Kreativität ein, obwohl man auch argumentieren kann, dass meist wiedererkennbare Strukturen generiert werden sollen, sodass die nicht wirklich ein Problem ist. Und durch geschickte Kombination bekannter Erdstrukturen kann viel neues erzeugt werden.

- Schwer weitere Regeln direkt einzubauen.
- Grundsätzlich ist es schwer dafür zu sorgen das Neuronale-Netzwerke nur valide Lösungen erzeugen, gerade wenn die Regeln komplex werden.

2.2 Lazy neu Errechnung

Dieser Teil betrachtet in wie weit die in Ansätze beschreiben in Abschnitt 2.1 Möglichkeiten zur Lazy Neuerrechnung beschreiben in Abschnitt 1.1 bieten.

ToDo:

Allgemein nach Arbeiten suchen.

2.2.1 Noise und Zufall

- Regeln ergeben sich aus Code also würde Regeln änder das der Code geändert wird.
- Individual Lösungen benötigt
- Teilweise werden generierte Ergebnisse aus alten Versionen in neuen weiter verwendet, aber sie entsprechen dann nicht den Regeln.
- Grundsätzlich benötigt man ein System welches die generations Regeln genauer modelliert um verallgemeinerbare Aussagen treffen zu können.

2.2.2 Lazy Model Systhesis

- Meine Erste Idee
- Man generiert ein valide Welt mit dem mit Model Systhesis
- Nun ändern sich die Regeln
- Man geht über alle Felder welche nicht valide Regeln haben.
- Nun müsste man in der Idee von Model-Systhesis wieder andere Werte zu diesen Feldern hinzufügen
- Aber welche? Alle?
- Das Hauptproblem: Die neu Hinzugefügten Werte sind erstmal auch nicht valide. Zu den Nachbarn müssten auch Werte hinzugefügt werden, damit sie valide sind.
- Das Problem gilt dann aber auch wieder für die Nachbarn. Und so weiter
- Wenn man einfach für alle Nachbarn alle Werte hinzufügt, hat man wieder das Komplett unentschiedene Feld normalen Model-Systhesis Algorithmus das bietet kein Vorteil.
- Also müsste man ein Menge an Feldern finden den man jeweils wieder Werte pro Feld hinzufügt, wo alle Regel eingehalten werden, damit mann dann wieder Model-Systhesis laufen lassen kann.
- Diese Menge müsste möglichst minimal sein.

- Vorallem sollte die Laufzeit-komplexität zum errechnen dieser Menge kleiner sein als Model-Systhesis selbst.
- Die einzige Idee die mir eingefallen ist um dies zu lösen:
  - Eine Breiten suche über ein Graph aller möglichen Werte die wir hinzufügen können. Um die erste valide Menge zu finden.
  - Das ist schrecklich was laufzeit und auch space Komplexität angeht.
- Der Vorteil von Model-Systhesis ist das in jedem Schritt alle Kombinationen an Werten eine valide Lösung ist.
- Aber diese Menge zu suchen ist sehr schwer.

2.2.3 KI basierte prozedurale Generation

- Schon alleine mehrere Ergebnisse die gleich aussehen zu errechnen ist schwer.

ToDo:

Aber vielleicht gibt es ja Arbeiten in die Richtung -> Nach Papern suchen

3 Theoretische Grundlagen

3.1 Träges errechnen (Lazy computation)

Träges errechnen auf englisch Lazy computation beschreibt die Idee in Computerprogrammen die Daten erst dann zu errechnen wenn sie benötigt werden.

Es reduziert Lag-spikes gerade beim starten eines neuen Prozesses, da Anstatt alle nutzbaren Daten nur die, die gerade angefragt benötigt werden errechnet werden.

Funktionale Programmiersprachen wie Haskell findet dieses Konzept vie Anwendung und erlaubt für unendliche Datenstrukturen.

3.2 Graphen

Die Datenstrukturen in dieser Arbeit basieren auf mathematische Graphen.

Daher erklärt dieser Abschnitt die in der weiteren Arbeit genutzten Notationen.

Ein Graph $G := (V, E)$ ist besteht aus einer Menge an Knoten V und einer Menge an Kanten E. Die Funktionen $V(G) = V$ und $E(G) = E$ werden als verkürzte Notationen genutzt.

Ein einzelner Knoten wird als $v \in V$ und eine einzelne Kante als $e \in E$ geschrieben.

Eine Kante ist ein Tuple zweier Knoten $e := (v_a, v_b)$. In dieser Arbeit sind Kanten grundsätzlich gerichtet und werden dann auch als $\overrightarrow{v_a v_b}$ geschrieben.

Die Menge aller eingehenden bzw. ausgehender Kanten eines Knoten v wird als $E_G^-(v)$ und $E_G^+(v)$ geschrieben.

Die Menge aller Knoten die eine eingehende bzw. ausgehende Kante zu einem Knoten haben werden als $N_G^-(v)$ und $N_G^+(v)$ geschrieben, da sie als die Nachbarn von diesem Knoten verstanden werden.

Diese Notationen basieren auf den Notationen in „Modern Graph Theory“ von Béla Bollobás [7].

ToDo:

CSG

ToDo:

Geometry Nodes

3.3 Abhängigkeiten

3.3.1 Prozedurale Generation aus der Design Sicht

- Rekursiv
- Abhängigkeiten werden schnell komplex
- Dependency Injektion

Die Entscheidung ob in einer prozeduralen Welt ein Objekt existiert, bzw wo es plaziert werden soll, hängt davon ab wie der Rest der Welt generiert wird.

Zum Beispiel wenn man einen Wald mit einem Netz an Wegen generieren möchte, könnte es mehrere Abhängigkeiten geben.

- Ein Baum sollte in Mindestabstand zu jedem anderem Baum haben.
- Ein Baum sollte ein Mindestabstand zu jedem Weg haben.
- Zwei Wege sollten nicht direkt nebeneinander laufen.

Nun stellt sich die Frage wie man dies effizient generieren kann.

Der üblichste Weg ist einen Algorithmus zu finden der iterativ die Bestandteile entscheidet. Der Algorithmus ist dabei so gewählt das er für jedes Bestandteil nur Ergebnisse erzeugt, die alle Abhängigkeiten einhalten.

An dem Beispiel würde als ersten Schritt ein Netz an Wegen generiert werden. Hierbei würde ein Algorithmus gewählt werden der intrinsisch alle benötigten Beschränkung einhält.

Dafür würde sich z.B. Passions Disk Sampling für die Position der Knoten anbieten, um einen Mindestabstand zwischen den Knoten sicherzustellen.

Nun könnten alle Knoten mit einem Abstand in einem bestimmten Bereich mit einem Weg verbunden werden. So könnten unrealistisch kurzen oder langen Wegen vorgebeugt werden.

Jedoch kann es Beschränkungen geben, die nicht intrinsisch von einem generations System eingehalten werden.

z.B. um Vorzubeugen, dass zwei Wege direkt nebeneinander parallel laufen, müssten im nach hinein unerwünschte Wege raus gefiltert werden.

Wurde nun so ein valides Weg Netz gefunden, kann dieses genutzt werden um Bäume nur an Stellen zu plazieren, wo kein Weg ist.

Dieses System zu generation von Welten ist sehr weit verbreitet. Dabei werden die Algorithmen und parameter so gewählt, das meist ein realistisches Ergebnisse erzeugt wird.

Jedoch da die Kern-Beschränkungen meist nur indirekt zugesichert werden, kann es zu Fehlerhaften Welten kommen.

ToDo:

Beispiel broken Minecraft Seeds

ToDo:

Doppelung mit state of art

3.4 Systeme, die alle Abhängigkeiten sicherstellen.

Notiz:

Das ist sehr wichtig

3.4.1 Linear Programming

ToDo:

Erklärung

3.4.2 Model Synthesis

ToDo:

Erklärung

3.4.3 Problem

- Zu schlechte Laufzeit Komplexität
- Zu schlechte Speicher Komplexität
- Können nur auch endlichen Mengen an Parametern arbeiten
  - Bei Model Synthesis wird diese durch Das Gitter und die Endliche Menge an Werten sichergestellt.

Man könnte jeden möglichen Zustand von jedem möglichen Objekt als ein Knoten in einem Graph modellieren. Wenn man nun eine Kante für jede Abhängigkeit zwischen zwei möglichen Zuständen definiert

4 Meine Arbeit

4.1 Kern Idee

Die Idee ist das die gesamte zu generierende Welt als Graph an Operationen definiert. Operationen errechnen aus eine Menge an Input Werten in eine Menge an Output Werten.

Beispiel für Operationen sind:

- Addition
- Erzeugung einer Kugel aus Position und Größe
- Union zweier Volumen
- etc

Operationen nutzen die Ergebnisse von anderen Operationen und bilden so ein Graph an Abhängigkeiten.

Bei der Errechnung werden die Ergebnisse der Operations Knoten zwischen gespeichert.

Wenn sich nun der Abhängigkeits-Graph minimal ändert entspricht das vorherige Ergebnis potenziell nicht mehr dem Abhängigkeits-Graph. Dies verstehe ich in dem Rest der Arbeit als eine invalide Lösung/Ergebnis.

Jedoch muss durch die zwischen Speicherung nicht den ganze Graph neu errechnet werden, sondern nur den Teil der nicht mehr valide ist.

Die Kern Leistung meiner Arbeit ist die Entwicklung einer effizienten Implementierung für dieses Modell.

4.2 Abhängigkeits-Graph

Jede mathematische Formel oder Algorithmus kann als Graph an Abhängigkeiten dargestellt werden. Dabei beschreibt jeder Knoten eine mathematische Operation die aus ein oder mehren Eingangswerten einen Ausgangswert errechnet. Die Eingangswerte eines Knoten sind die Ergebnisse andere Knoten im Graph. Dies bezeichne ich als Abhängigkeit, da die Ergebnisse der Eingangsknoten für die Berechnung benötigt werden. Das Ergebnis eines Knoten hängt nur von den Eingangswerten ab und hat damit keine Nebeneffekte. (siehe Funktionale Programmierung) Knoten können keine Eingangswerte haben dann erzeugen diese immer die gleichen Werte und werden als konstant verstanden.

4.3 Framework

ToDo:

Overview diagramm

Mein generations System besteht aus drei Bestandteilen.

1. Ein Graphischer Editor indem ein Nutzer einen Abhängigkeits-Graph erstellen und bearbeiten kann.

2. Das Template ist eine Datenstruktur die vom Editor erstellt wird und den Abhängigkeits-Graph sowie Informationen wie dieser generiert und zwischen gespeichert werden soll enthält.
3. Der Generator vergleicht das aktuelle Template mit dem neuen Template und generiert die Bestandteile der Welt neu die nicht dem neuen Template entsprechen.

4.4 Composer

4.5 Template

Das Template $:= (G_{\text{ab}}, G_{\text{ch}})$ besteht aus dem Abhängigkeits-Graph G_{ab} und einem „cache“ Graphen G_{ch}.

G_{ab} ist ein Graph der die zu generierende Welt als rekursive Formel an Operationen beschreibt.

Die ein gehenden Nachbarn $N^-_{G_{\text{ab}}}$ errechnen die Eingangswerte für eine Operation und die ausgehenden Nachbarn $N^+_{G_{\text{ab}}}$ sind alle Operationen die das Ergebnis benötigen.

ToDo:

Beispiel?

G_{ch} enthält einen Knoten für jeden Knoten in G_{ab} der Zwischengespeichert werden soll. Dies ist eine Untermenge aller Knoten in G_{ab}.

$V(G_{\text{ch}}) \subset V(G_{\text{ab}})$

Bei der Entscheidung wie groß diese Untermenge sein soll muss zwischen dem „Overhead“ durch zwischengespeicherung und der Zeitersparniss durch wiederverwendung der Ergebnisse abgewägt werden.

Die ein gehenden Nachbarn $N^-_{G_{\text{ch}}}$ sind alle caches von dem der Knoten abhängt. Man findet diese indem man den Baum der Abhängigkeiten in G_{ab} über alle Verzweigungen rekursiv durchsucht bis man auf jeden Weg einen auf einen Knoten in G_{ch} stößt.

ToDo:

Algorithmus angeben? eigentlich ist das nicht wichtig.

ToDo:

Grafik von Datentypen und Abhängigkeits-Graph

4.5.1 Level

Wenn der G_{ab} ein directed acyclic Graph (DAG) ist dann kann jedem Knoten $v \in G_{\text{ab}}$ ein Level $l(v)$ zugeordnet werden. Dies ist definiert als:

$$l(v) > l(v_i) \quad \forall v_i \in N^-_{G_{\text{ab}}}(v)$$

Also das Level eines Knoten ist immer größer als das Level aller Knoten von den er abhängt.

Um den G_{ab} zu errechnen werden die Knoten der Level aufsteigend errechnet. Die Rheinfolge innerhalb eines Levels ist nicht relevant.

4.5.2 Einen Prozeduralen Algorithmus als Template darstellen

Bis zu diesem Abschnitt habe ich das Template sehr allgemein beschrieben.

Um mit dem Template eine Prozedurale Welt zu generieren haben ich mich entschieden, dass das Template final eine Volumen als „constructive solid geometry“ (CSG) errechnet.

Diese CSG setzt sich durch union und remove Operationen auf primitiven Geometrien wie Kugeln und Boxen zusammen.

Daher sind die Operationen die ich implementiert habe:

- CSG Union und Remove
- Kugel aus Position und Durchmesser
- Box aus Position und Seitenlänge
- Alle Positionen auf einem Gitter die innerhalb eines Volumen sind.
- Eine Menge an zufälligen Positionen innerhalb eines Volumen. siehe: genauer dazu
- Addition, Subtraktion, Multiplikation und Division von Positionen und Zahlen
- etc.

4.5.3 Generation eines Templates

Operationen im Abhängigkeits-Graph G_{ab} können auch Mengen an Werten erzeugen. In meiner implementation ist das die beiden Operationen die ein Gitter und zufällige Positionen in einem Volumen errechnen.

Nun kann es sein, dass die weiteren Operationen pro Element in dieser Menge ausgeführt werden sollen. Dies ist der Bestandteil von prozeduraler Generation der es möglich immer feinere Details zu generieren. Zu Beispiel errechnet man eine Menge an Positionen an den Apfelbäume stehen sollen und dann generiert man pro Baum die Positionen der Äpfel.

Dies ergibt einen klaren Unterschied in der Laufzeit von Algorithmen auf dem Template zu Algorithmen auf der generierten Welt. Die Menge an Knoten im Abhängigkeits-Graph und so auch den cache Graph skaliert mit der Menge an Operationen des Generationsalgorithums. Wohin die Menge der Elemente in der Welt mit dem Größen der Mengen an rechneten Werten skaliert. In anderen Worten alle Knoten im Template zu iterieren ist relativ schnell möglich wohin gegen die Laufzeit alle Elemente in der Welt zu iterieren exponentiell steigen kann.

Daher arbeiten alle Algorithmen im meinen System immer nur auf den Knoten des Templates. Sie nutzen die Abhängigkeiten im Template um heraus zu finden wie die Welt neu generiert werden muss.

4.6 Generator

Der Generator enthält einen Graphen G_{gen} der den cache Graphen G_{ch} im Template entspricht. Jeder Knoten $v_{\text{gen}} \in V\big(G_{\text{gen}}\big)$ speichert welchem Knoten $v_{\text{ch}} \in V(G_{\text{ch}})$ er ent-

spricht $v_{\text{ch}} = \textbf{cache}(v_{\text{gen}})$. Dazu hat ein Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ das Level des entschieden Template Knoten $l(v_{\text{gen}}) = l(\textbf{cache}(v_{\text{gen}}))$.

Jedoch wo G_{ch} nur einen Knoten pro Operation enthält, enthält G_{gen} einen Knoten pro Ergebnis welches errechnet werden muss.

ToDo:

Beispiel

Dazu hält der Generator $:= (G_{\text{gen}}, Q_{\text{tasks}})$ eine Queue Q_{tasks} die zwei Arten Aufträge auf G_{gen} nach ihren Level sortiert.

$$\text{pop}(Q_{\text{tasks}}) := \min_{q \in Q_{\text{tasks}}} (l(q))$$

Errechnungs-Aufträge errechnen den das Ergebnis eines Knoten in G_{gen} und Kind-Update-Aufträge erzeugen oder löschen Kinder sodass sie dem Template entsprechen.

4.6.1 Abhängigkeite Werte im Generator Graph finden

Um einen Knoten in G_{gen} zu errechnen benötigt man die Ergebnisse der Knoten in G_{gen} von den dieser Knoten abhängt. Wie ich in Abschnitt 4.5.3 erklärt habe ist es Laufzeittechnisch nicht möglich diese mit z.B. einer Tiefensuche zu suchen.

Stattdessen wird für jeden Template Knoten $v \in V(G_{\text{ch}})$ einer der Knoten von den er abhängt $N^-_{G_{\text{ch}}}(v)$ als Erstellungs-Knoten $v_c \in V(G_{\text{ch}})$ im Template markiert $v_c = \textbf{create}(v)$.

Um nun für einen Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ alle anderen Knoten zu finden von er abhängt $N^-_{G_{\text{gen}}}(v_{\text{gen}})$, werden die relative Schritte in G_{ch} vom Erstellungs-Knoten v_c zu den anderen Abhängigen Knoten als Baum gespeichert $T_{\text{rel}}(v_{\text{gen}})$.

Ein relativer Schritt v_{step} gibt entweder an das man von einem Knoten $v \in V(G_{\text{ch}})$ hoch ($\textbf{up}(v_{\text{step}}) = \text{True}$) in einen Knoten $v_{\text{up}} \in V(G_{\text{ch}})$ von dem v abhängt ($v_{\text{up}} \in N^-_{G_{\text{ch}}}(v)$) oder runter ($\textbf{up}(v_{\text{step}}) = \text{False}$) in einen Knoten $v_{\text{down}} \in V(G_{\text{ch}})$ der von v abhängt ($v_{\text{down}} \in N^+_{G_{\text{ch}}}(v)$) gehen soll.

Da ein Knoten $v \in V(g_{\text{ch}})$ mehr als einen eingehenden oder ausgehenden Nachbarn haben kann speichert ein relativer Schritt auch in welchen Nachbarn gegangen werden soll ($\textbf{cache}(v_{\text{step}})$) und dazu ob dieser Nachbar eine Abhängigkeit für v_{gen} ist ($\textbf{deps}(v_{\text{step}}) = \text{True}$).

Jeder relative Schritt v_{step} gibt speichert auch in welche der Nachbar Knoten gegangen werden soll.

Diese relativen Schritte nutzen nur Knoten die ein kleineres Level haben als v_{gen}. Da im Generator Knoten im Level aufsteigend erstellt werden, ist so sichergestellt das alle relativen Wegen existieren.

Für einen Knoten im Template kann es mehrere Knoten im Generator geben, daher können pro Abhängigkeit eines cache Knoten auch mehrere Knoten im Generator gefunden werden.

Algorithm 1: Finde abhänige Knoten in G_{gen}

```
procedure FindDeps(v_gen, v_gen creates)
    T ← T_rel(v_gen)
    D ← ∅
    Q ← ∅
    push(Q , (v_root ∈ T, v_gen creates))

    while Q ≠ ∅ do
        (v_step, v_gen) ← pop(Q)

        if deps(v_step) then
            push(D , v_gen)
        end

        for v_child step ∈ N_T^+(v_step) do
            N ← N_{G_gen}^-(v_gen) if up(v_child step) else N_{G_gen}^+(v_gen)

            for v ∈ N do
                if cache(v_child step) = cache(v) then
                    push(Q, (v_child step, v))
                end
            end
        end
    end
    return D
end
```

ToDo:

Beispiel Zeichung

4.6.2 Kind-Update-Aufträge

Kind-Update-Aufträge enthalten den Index des Erstellungs-Knoten und den Index eines Erstellungs-Eintrag $E_{\text{create}}(v_{\text{ch}})$ in dessen Template-Knoten. Dieser Erstellungs-Eintrag definiert wie viele Kinder es geben soll $\textbf{num}(v_{\text{ch}}, v_{\text{gen creates}})$. Dies sind entweder es genau n pro Erstellungs-Knoten oder, oder hängen von dem Wert des Erstellungs-Knoten $v_{\text{gen creates}}$ ab wie z.B. einer Positions Menge. Dazu gibt $\textbf{valid}(v_{\text{gen}}, v_{\text{gen creates}})$ an ob ein Kind v_{gen} noch für den Erstellungs-Knoten $v_{\text{gen creates}}$ noch valide ist. z.B. ob eine Position noch in der Menge an Positionen ist.

Darauf wird die vorhandene Menge an Kindern mit der gewünschten Menge verglichen und bei Ungleichheit neue Kinder Knoten erzeugt oder gelöscht.

Wenn eine neuer Knoten erzeugt wird werden mit dem Baum an relativen Schritten die Indexe von allen abhängige Knoten gesucht und im Knoten gespeichert.

Algorithm 2: Kinder updaten

```
procedure UpdateChild(v_gen, v_ch child)

    C ← { v ∈ N⁺_{G_gen}(v_gen) | cache(v) = v_ch child }

    C_to delete ← { v ∈ C | ¬ valid(v, v_gen) }

    for v ∈ C_to delete do

        Delete(v)
    end

    n ← num(v_ch child, v_gen)
    i ← 0
    for i < n − |C \ C_to delete| do

        Create(v_ch child, v_gen)
    end
end

procedure Create(v_ch, v_gen creates)
    push(V(G_gen), v_new)

    cache(v_new) ← v_ch

    D ← FindDeps(v_new, v_gen creates)

    N⁻_{G_gen}(v_new) ← D

    for v ∈ D do
        push(N⁺_{G_gen}(v), v_new)
    end

    pushCalculate(Q_task, v_new)
end

procedure Delete(v_gen)

    for v ∈ N⁻_{G_gen}(v_gen) do

        remove(N⁺_{G_gen}(v), v_gen)
    end

    for v ∈ N⁺_{G_gen}(v_gen) do

        Delete(v)
    end
    remove(V(G_gen), v_gen)
end
```

4.6.3 Errechnungs Aufträge

Errechnungs Aufträge errechnen den Wert von einem Knoten $v_{\text{gen}} \in V(G_{\text{gen}})$ neu. Dabei wird der Knoten im Abhängigkeits-Graph rekursiv errechnet.

Wenn ein der Algorithmus auf einen Knoten $v_{\text{ab}} \in V(G_{\text{ab}})$ stößt, welcher ein cache Knoten hat $v_{\text{ab}} \in V(G_{\text{ch}})$ werden die Werte der jeweiligen abhängigen Knoten von v_{gen} verwendet.

ToDo:

Beispiel

4.6.4 Abhängigkeits Kreise

Das Template kann Abhängigkeits Kreise enthalten. Um trotzdem eine valide Lösung errechnen zu können muss es für jeden Knoten v_{val} einen validen Null Wert geben.

So kann der Abhängigkeits Graph iterativ gelöst werden. Dazu werden pro Kreis im Abhängigkeits-Graph eine Kante als durchgeschnitten markiert $N^{+}_{\text{cut}}(v) \subseteq N^{+}_{G_{\text{dep}}}(v) \quad v \in V\big(G_{\text{dep}}\big)$. Der Abhängigkeits-Graph ohne die durch geschnittenen Kanten $N^{+}_{\text{not cut}}(v) := N^{+}_{G_{\text{dep}}}(v) \setminus N^{+}_{\text{cut}}(v)$ ist ein DAG. Also kann jedem Knoten ein Level $l(v)$ zu geordnet werden.

$$l(v) > l(v_i) \quad \forall v_i \in N^{+}_{\text{not cut}}(v)$$

Die Knoten werden Level nach Level erzeugt und so sichergestellt das alle nicht geschnittenen Abhängigkeiten schon errechnet worden sind, wenn der Knoten selbst errechnet wird. Hat ein Knoten geschnittene Abhängigkeiten werden diese zu Errechnung genutzt wenn sie existiert. Andernfalls wird der Null Wert verwendet. Jeder Knoten der Null Werte für seine geschnittenen Abhängigkeiten nutzt wird nochmal errechnet, sobald alle Knoten einmal errechnet wurden. Dies wird so lange wiederholt bis keine Null Werte mehr verwendet worden sind.

4.7 Implementation

Für die Beispiel Implementation habe ich Rust als Sprache gewählt, da sie erlaubt speichersicheren low level code zu schreiben um die Laufzeit von Algorithm effektiv zu verbessern und ein in Gegensatz zu C++ ein umfassenden leicht zu nutzenden Package-Manager hat.

4.7.1 Stabiele Listen

Alle Graphen sind mit stabilen Listen implementiert.

Eine stabile Listen ist eine sich automatisch vergrößerndes Array. Wenn dass aktuelle Array voll ist wird ein größeres Array alloziert und die werte mit einer memcopy Operation rüber kopiert. Dazu änderen sich die Indexe von Elementen in stabilen Listen nicht. Wenn ein Element entfernt wird, werden die weiteren Elemente nicht aufgeschoben um die Lücke zu schließen, stattdessen wird der Index des nächsten freien Element gespeichert. Dazu speichert die List speichert den Index des ersten freien Element, welches genutzt wird wenn ein neues Element eingefügt wird. Der dort gespeicherte nächste freie Index wird dann als erster freier Index gespeichert. Dies erlaubt Einfüge-, Entfernungs- und Zugriffslaufzeit von $O(1)$.

Dazu wird pro Element eine Versionsnummer gespeichert. Bei einem Zugriff wird die Version des Index mit der Version des Elements an der Stelle des Index verglichen. So bleiben Indexe valide solange das Element in der Liste ist und es wird erkannt wenn man mit veralteten Index zugreift.

Die oberen 32Bit des Indexes werden genutzt um Versionsnummer zu speichern. Somit können in einer stabilen List 2^{32} Elemente gespeichert werden.

Stabiele Listen erlauben Graphen effizient als Listen darzustellen, indem in den Knoten die Indexe der anderen Knoten gespeichert werden zu den Kanten existieren. Stabiele listen sind dabei wesentlich schneller als HashMaps.

ToDo:

Quelle

4.7.2 Multi Threading

Der Collapser sowie die Sampeling Operationen werden in asynchronous Workern ausgeführt. Zur Kommunikation werden Channel verwendet.

ToDo:

Quelle

Der Composer läuft im Render Thread und errechnet bei jeder Änderung das aktuelle Template. Dieses Template wird über ein Channel zum Collapser gesendet. Wenn der Channel noch ein altes Template enthält werden die Änderungen Notizen des alten Template zum neuen hinzugefügt und das Template wird ersetzt.

Die errechnete CSG Darstellung der Welt wird mit einem weiteren Channel an die Sampler gesendet, welche dieses in ein Voxel DAG oder Mesh umrechnet und auf die GPU gelanden.

Wenn die neue Welt hochgenanden wurde wird der Render Thread über ein Channel über diese Änderung informiert.

4.7.3 Small Vectors

Für die Zwischenspeicherung der Werte werden Small Vectors verwendet. Diese haben die Eigenschaft, dass die ersten N Elemente direkt auf dem Stack alloziert werden. Erst wenn diese voll sind wird ein Array auf dem Stack alloziert.

Da alle Werte müssen als Liste behandelt werden, da ein Knoten immer von mehreren Knoten für ein Input abhängt. Jedoch enthält diese Listen meist doch nur ein Element. Small Vectoren erlauben für diese Fälle die Stack allozieren zu sparen und bieten bessere Cache Lokalität.

4.7.4 Output Datenstruktur

Das Kernconzept, einen sehr großen Graphen der eine prozeduralen Algorithmus darstellt, zu bearbeiten, indem man die Abhängigkeiten in einem vergleichbar kleinen Template nutzt, enthält keine konkreten Anahmen über die Art der Geometrie. In

meiner Implementation habe ich CSG genutzt, da es einer sehr allgemeine Form ist, Volumen darzustellen und diese leicht zu bearbeiten sind.

Gerade CSGs mit vielen Knoten sind jedoch nicht performant zu rendern. Deshalb können die CSGs in meiner Implementation entweder als Sparse Voxel DAGs oder mit marching cubes gesampled werden.

4.8 Minimales Beispiel

in welchem lazy generation einen klaren Vorteil hat.

ToDo:

Diagram

4.9 Nutzungs Beispiele

ToDo:

Bilder

- Welt editor
- Infitie World Generation
- LOD systems

5 Bewertung

5.1 Bewertung und Vergleich zu andere Systemen

5.1.1 Wie kann man überhaupt das System gut vergleichen?

5.1.1.1 Gleiche Regeln in beidem Systemen implementieren und dann Geschwindigkeit vergleichen

- Probleme
  - Ich habe nur meine Implementation.
  - Gibt es überhaupt Regeln für Welten die man in meinem System und WFC darstellen kann?
    - Warscheinlich wären die Regeln biased.
    - Regeln von vorhanden Spielen nutzen?

5.1.1.2 Nutzerfreundlichkeit bewerten

Sollte in meinem Fall im Bereich von gesunden Menschenverstand sein und daher wird keine Studie benötigt.

6 Future Work

6.1 Nullwerte führen zu leeren Lösungen

Da der Null Wert per definition eine valider Wert ist kann es dazu kommen, dass sich ein Abhängigkeits-Kreis zu null als Lösung entwickelt, auch wenn es theoretisch andere Lösungen gebe. Um dies zu lösen müsste ein anderer Ansatz zur Lösung von Abhängigkeits-Kreisen genutzt werden.

Arbeiten in Richtung closely connected Components in Verbindung könnten hier eine Lösung sein.

6.2 Andere Datenstrukturen

Wie schon in Abschnitt 4.7.4 beschrieben eignen sich CSGs nicht zum direkten rendering. Die Umwandlung zu einem Mesh mit „marching cubes“ ist aufwendig und mehr wesentlich mehr Leistung benötigen, als die Generation selbst. Somit besteht weiterhin die Forschungsfrage in wie weit minimale Neuberechnung von prozeduralen Meshen möglich ist.

Bibliografie

[1] P. Merrell, „Example-based model synthesis“, in *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games*, in I3D '07. Seattle, Washington: Association for Computing Machinery, 2007, S. 105–112. doi: 10.1145/1230100.1230119.

[2] V. Volz, J. Schrum, J. Liu, S. M. Lucas, A. Smith, und S. Risi, „Evolving Mario Levels in the Latent Space of a Deep Convolutional Generative Adversarial Network“. [Online]. Verfügbar unter: https://arxiv.org/abs/1805.00728

[3] M. Bazzaz und S. Cooper, „Level Generation with Constrained Expressive Range“, in *Proceedings of the 20th International Conference on the Foundations of Digital Games*, in FDG '25.: Association for Computing Machinery, 2025. doi: 10.1145/3723498.3723845.

[4] O. Withington und L. Tokarchuk, „Compressing and Comparing the Generative Spaces of Procedural Content Generators“, in *2022 IEEE Conference on Games (CoG)*, 2022, S. 143–150. doi: 10.1109/CoG51982.2022.9893615.

[5] A. Goslin, „Terrain Diffusion: A Diffusion-Based Successor to Perlin Noise in Infinite, Real-Time Terrain Generation“. [Online]. Verfügbar unter: https://arxiv.org/abs/2512.08309

[6] J. Whitehead *u. a.*, „Conversational Interactions with Procedural Generators using Large Language Models“, in *Proceedings of the 20th International Conference on the Foundations of Digital Games*, in FDG '25.: Association for Computing Machinery, 2025. doi: 10.1145/3723498.3723788.

[7] B. Bollobás, *Modern Graph Theory*. Springer New York, 2013. doi: 10.1007/978-1-4612-0619-4.

Idee:

Text

ToDo:

Text

TODO: Text

Frage:

Text

$$A = \pi \tag{1}$$